

CrimeLens Prototype - Software Design Document

Sep 19, 2026 · @Someone

1. Document overview

This document specifies the design of the CrimeLens prototype for Smart India Hackathon 2026 (problem statement 26189, team Cyber Titans). It states what the prototype does, how it is built, and which details the team has not yet decided.

Inputs. Two team documents are the only sources: the presentation ("CrimeLens", 6 slides) and the detailed solution proposal (titled "NETRA-X", 42 sections). Nothing here comes from anywhere else.

Naming. The presentation calls the product CrimeLens and the proposal calls it NETRA-X. This document uses CrimeLens and treats the proposal's content as the working specification (see D1).

How gaps are handled. Where a source states a detail, this document states it too. Where neither source does, the text says **Decision needed** with an ID (D1 to D24) that points to the table in section 15. No default has been invented.

Marker	Meaning
Plan	Stated in the presentation or the proposal
Derived	A direct consequence of a stated rule, not a new choice; labelled wherever used
Decision needed (Dn)	Not stated in either source; the team must decide (section 15)

Audience. Team members building the prototype, and mentors or judges reviewing it.

2. Prototype scope

The prototype is a batch-processing, synthetic-data investigation platform run with Docker Compose. The proposal ranks 16 features MUST, 3 NICE and streaming, GNN and distributed graph FUTURE.

2.1 Feature scope

Feature	Plan priority	Designed in
RBAC login	MUST	12
Case creation	MUST	11
Evidence upload	MUST	5
Synthetic dataset generator	MUST	5
NER / entity extraction	MUST	6

Feature	Plan priority	Designed in
Entity resolution	MUST	7
Neo4j graph	MUST	8
Interactive graph	MUST	11
Centrality and community analysis	MUST	9
Temporal timeline	MUST	9, 11
Anomaly detection	MUST	9
Evidence provenance	MUST	8
Explainable priority leads	MUST	9, 10
Grounded copilot	MUST	10
Report generation	MUST	10
Audit logs	MUST	12
Contradiction detection	NICE	9
Geo-temporal map	NICE	11
Mission mode	NICE	11 (no design detail in the plan)
Streaming ingestion, GNN, distributed graph	FUTURE	Out of scope
Hidden-link discovery	Not in the priority table (D2)	9
Network evolution ("What Changed?")	Not in the priority table (D2)	9
Landing screen	Not in the priority table (D2)	11

The last three rows appear in the proposal's innovation portfolio, demo script, checklist or metrics, but not in its MUST / NICE / FUTURE table. They are designed here as written in the plan; their priority is decision D2.

2.2 Out of scope (Plan)

- Streaming ingestion with Kafka, Spark or Polars processing, OpenSearch or Elasticsearch, S3-compatible object storage, Kubernetes.
- node2vec / GNN hidden-link models, Leiden or dynamic communities, a fine-tuned multilingual transformer.
- Live integration with police systems. The prototype demonstrates the integration contract only.

- Real personal data. The demo uses synthetic data, plus legally reusable public research data where licences permit.

2.3 Design rules (Plan)

1. A relationship is never just an edge. Every relationship carries source, timestamp, evidence reference, confidence and reliability.
2. Every finding is labelled FACT, INFERENCE or RECOMMENDATION.
3. The system assists investigators and does not decide guilt. It reports "Investigation Priority", never a criminal probability.
4. Conflicting claims are preserved with their sources and flagged for review, never auto-resolved.
5. The copilot is read-only and answers only from retrieved evidence. When evidence is missing it says so.
6. Uncertainty and missing data are shown. Missing data is never treated as proof of absence.

2.4 Constraints (Plan)

- CPU-first. A GPU is optional and only speeds up transformer inference.
- Minimum demo hardware: 4 to 6 CPU cores, 16 GB RAM, 50+ GB SSD.
- Recommended development hardware: 8+ cores, 32 GB RAM, optional 8 to 12 GB VRAM GPU, 100+ GB SSD.

3. System overview and architecture

CrimeLens runs a staged pipeline that writes to two stores, then analyses the graph and serves the results through a FastAPI API to a Next.js UI. This follows the architecture in proposal section 19 and the seven-step flow on presentation slide 3.

flowchart TD

```

A["Data sources<br/>FIR, CDR, finance, reports, OSINT"] --> B["Ingestion + validation"]
B --> C["NLP extraction"]
C --> D["Entity resolution"]
D --> E["Evidence + provenance"]
E --> F["PostgreSQL<br/>cases, evidence, audit"]
E --> G["Neo4j<br/>temporal graph"]
F --> H["Anomaly + temporal engine"]
G --> H
H --> I["Explainable lead engine"]
I --> J["FastAPI intelligence API"]
J --> K["Next.js UI<br/>graph, map, timeline, copilot"]
K --> L["Reports + audit"]

```

Data moves top to bottom. PostgreSQL holds users, cases, evidence metadata and audit records. Neo4j holds the graph used for traversal, paths and community analysis.

3.1 Components

Component	Responsibility (Plan)	Designed in
Ingestion and validation	Collect data in structured and unstructured formats; cleanse,	5

Component	Responsibility (Plan)	Designed in
	normalise and validate it	
NLP extraction	Extract entities, events and relations from text	6
Entity resolution	Merge aliases and inconsistent formats into canonical entities	7
Evidence store (PostgreSQL)	Users, cases, documents, evidence metadata, notes, audit	8, 12
Graph store (Neo4j)	Entities and relationships with provenance and time	8
Anomaly and temporal engine	Anomalies, window comparison, network evolution	9
Lead engine	Explainable investigation priority and hidden-link leads	9
Copilot	Read-only, evidence-grounded natural-language queries	10
Report generator	Intelligence brief with facts, inferences and evidence	10
FastAPI API	Typed contracts (Pydantic) between UI and services	11
Web UI	Command centre, case workspace, graph, timeline, entity profile, copilot, reports, audit	11

3.2 Processing mode

The prototype processes data in batches (Plan: MVP ingestion is "Batch CSV/JSON/PDF/text", processing is Python/Pandas). How an ingestion run is triggered and executed, for example inside the API request or as a background job, is not specified (D17).

4. Technology stack

The prototype uses open-source technologies only. The presentation shows 12 of them; the proposal adds Tailwind CSS, shadcn/ui, Transformers, Neo4j GDS, NetworkX, Pydantic and Docker Compose.

Layer	Technology	Role in the prototype	Source
Frontend	Next.js, React,	Web application	Both

Layer	Technology	Role in the prototype	Source
	TypeScript		
UI components	Tailwind CSS, shadcn/ui	Consistent interface	Proposal
Graph visualisation	Cytoscape.js	Interactive graph and layouts	Both
Maps	MapLibre GL JS or Leaflet	Geospatial view (NICE)	Both; library choice open (D23)
Backend	FastAPI, Python	API and ML/data workflows	Both
API validation	Pydantic	Typed API contracts	Proposal
Relational database	PostgreSQL	Users, cases, evidence metadata, audit	Both
Graph database	Neo4j	Traversal, paths, communities	Both
NLP	spaCy, Transformers	NER and relation extraction	Both (Transformers: proposal)
Machine learning	scikit-learn	Anomaly baselines (Isolation Forest)	Both
Graph analytics	Neo4j GDS, NetworkX	Centrality, communities, paths	Proposal; split between them open (D23)
Testing	pytest, Playwright	Backend and end-to-end tests	Both
Deployment	Docker Compose	Reproducible setup	Proposal
LLM	Controlled API or local model, "as permitted"	Copilot	Proposal; model not chosen (D3)

Versions of these components are not specified in either source.

4.1 Algorithms and methods named for the prototype (Plan)

Task	Prototype method	Reference cited in the sources
Named-entity recognition	spaCy or Transformer NER	None
Relation extraction	Rules, patterns and a classifier	None
Entity resolution	Normalisation, fuzzy matching, identifiers, graph context	Fellegi and Sunter (1969)

Task	Prototype method	Reference cited in the sources
Centrality	Degree, betweenness, closeness, eigenvector, PageRank	None
Community detection	Louvain	Blondel et al. (2008)
Anomaly detection	Isolation Forest with baselines	Liu, Ting and Zhou (2008)
Hidden links	Common neighbours, paths, temporal evidence	None
Copilot	LLM with retrieval and safe query templates	Lewis et al. (2020), retrieval-augmented generation

The proposal also lists Lundberg and Lee (2017) on model explanation. It does not say whether the prototype uses SHAP.

5. Data sources and ingestion

The prototype ingests batch files in CSV, JSON, PDF and plain text, drawn from five source types, and demonstrates them with a synthetic dataset whose ground truth is known.

5.1 Source types and formats (Plan)

- Source types: FIR, CDR, finance / transactions, surveillance, OSINT. The proposal also names reports and case history.
- MVP formats: batch CSV, JSON, PDF and text, both structured and unstructured.
- Uploaded evidence is stored per case with a file hash (documents table, section 8).
- Uploaded text is untrusted data and is never treated as instructions.

5.2 Pipeline stages (Plan)

Raw, clean, normalise, anonymise, extract, resolve, relate, timestamp, confidence, graph, benchmark. The presentation groups the first three as "Ingest & Validate": cleanse, normalise and validate for consistency, quality and reliability.

The validation rules themselves (required fields, formats, rejection behaviour) are not specified (D17). The anonymisation method is not specified (D19).

5.3 Record schemas (Plan)

Record	Fields
FIR	fir_id, case_id, date, location, narrative, source, language
CDR	cdr_id, caller, receiver, timestamp, duration, cell_id

Record	Fields
Transaction	txn_id, sender, receiver, timestamp, amount, channel, reference
Person	person_id, name, aliases, identifiers, attributes
Vehicle	vehicle_id, registration, type, owner_ref
Location	location_id, coordinates or area, label
Case	case_id, category, opened_at, status, investigator
Relationship	rel_id, from_id, to_id, type, timestamp, source, confidence

No schema is given for surveillance notes, OSINT, organisations, bank accounts, social-media accounts, events or documents. Field types, units and the mapping from CDR cell_id to a location are also unspecified (D17).

5.4 Synthetic dataset generator (MUST)

The generator produces ground-truth-controlled data so that entity resolution, hidden-link discovery and anomaly detection can be scored objectively.

Layer	Content	Purpose
Synthetic ground truth	People, aliases, calls, transactions, locations, vehicles, cases	Primary end-to-end demo
Synthetic unstructured text	FIR-like narratives, surveillance notes, reports	NLP demonstration
Public research data	Open or anonymised datasets, only where licences permit	External validation
Benchmark split	Train, validation and test with planted links and anomalies	Objective evaluation

Network	People	Phones	Locations	Vehicles	Transactions
A	25	8	5	4	15+
B	12	4	3	2	10+

Eight patterns are planted: bridge node, communication spike, transaction chain, alias, co-location, new cross-community edge, conflicting evidence, anomaly.

Not specified: the number of calls, FIRs and narratives, the time span, the incident date, the random seed, the split ratios (D21), and which public datasets and licences are used (D20).

5.5 Data-handling rule (Plan)

The public demo uses synthetic or pseudonymised data. It does not need private CDRs or bank records.

6. NLP extraction pipeline

The NLP stage turns narratives and records into candidate entities, events and relationships, using spaCy or Transformer NER plus rules, patterns and a classifier for relations.

6.1 What the stage does (Plan)

Task	Prototype method	Output
Named-entity recognition	spaCy or Transformer NER	Entity spans and types
Relation extraction	Rules, patterns and a classifier	Candidate relationships

The presentation states the stage works on structured and unstructured text. The proposal lists these extraction targets: people, organisations, phones, vehicles, locations, accounts and events.

6.2 Output requirements (Plan)

- Every extracted relationship carries source_record_id, timestamp, confidence, reliability, evidence_class and metadata (section 8).
- Model name and version are stored with results for reproducibility.
- Extracted candidates go on to entity resolution (section 7). They are not written to the graph as final entities first.

The second bullet's storage location is not specified (D16).

6.3 Security (Plan)

Input text is treated as untrusted data. Instructions found inside documents are never executed.

6.4 Quality target (Plan)

The proposal sets a target of 0.85 or higher on a controlled domain set. It does not say whether this applies to F1 alone or to precision, recall and F1. Results must be reported as measured, not as achieved (section 13).

6.5 Not specified

- Which spaCy pipeline or Transformer model is used, and whether any model is fine-tuned (D4).
- Which languages are supported. The FIR schema has a language field, and the proposal mentions language detection and multilingual models, but lists a fine-tuned multilingual transformer only as roadmap (D4).
- The relation label set, the classifier type and its training data (D5).
- The list of event types (D5).

7. Entity resolution

Entity resolution merges aliases and inconsistent formats into canonical entities, and sends uncertain matches to a human instead of merging them.

7.1 Method (Plan)

The prototype method is normalisation, fuzzy matching and identifiers. The proposal's answer to "How does entity resolution work?" adds graph evidence: normalise identifiers, generate candidates with fuzzy matching, then use contextual identifiers and graph evidence before merging or flagging. Embedding-based resolution is roadmap only.

```
flowchart LR
  A["Normalise identifiers"] --> B["Fuzzy candidate pairs"]
  B --> C["Contextual identifiers<br/>+ graph evidence"]
  C --> D{"Confident match?"}
  D -- "yes" --> E["Merge + keep merge provenance"]
  D -- "uncertain" --> F["Flag for human review"]
  F --> G["Authorised reviewer confirms or corrects"]
```

The test for "confident" is not stated (D6).

7.2 Behaviour requirements (Plan)

- Merge provenance is kept for every merge.
- Authorised users can correct entity-resolution mistakes, including a wrongly merged person.
- A record that maps one alias to two different people is flagged as identity ambiguity, not silently resolved (section 9.5).
- The failure modes to control are false merges (two people shown as one node) and false splits (one person shown as several nodes).

7.3 Quality target (Plan)

Precision of 0.90 or higher on the synthetic benchmark, with recall and F1 also reported. Ground truth comes from the planted aliases (section 5.4). This is a target, not a measured result.

7.4 Not specified (D6)

- The normalisation rules per identifier type (names, phones, vehicle registrations, accounts).
- The fuzzy-matching algorithm and similarity thresholds.
- How graph evidence is used in the merge decision.
- Whether the Fellegi and Sunter (1969) record-linkage model, cited in the presentation, is implemented or is only background.
- Where merge provenance and review decisions are stored (D16).
- Whether resolution runs within one case or across cases (D13).

8. Evidence store and temporal knowledge graph

PostgreSQL stores cases, evidence metadata and audit records; Neo4j stores 12 node types and 11 relationship types, and every relationship carries its provenance.

8.1 Graph schema (Plan)

Node types (12): Person, Organization, Phone, Vehicle, Location, BankAccount, Transaction, Case, FIR, Event, SocialMediaAccount, Document.

Relationship types (11): CALLED, TRANSFERRED, VISITED, OWNED, ASSOCIATED_WITH, WORKED_FOR, LOCATED_AT, MENTIONED_IN, PART_OF, CONNECTED_TO, PARTICIPATED_IN.

Edge property	Example	Purpose
source_record_id	CDR-023	Evidence trace
timestamp	2026-08-11 09:10	Temporal analysis
confidence	0.94	Model or data confidence
reliability	0.90	Source reliability
evidence_class	OBSERVED	Fact versus inference
metadata	duration=1020s	Context

The proposal's example edge, verbatim in structure:

```
(:Person {id:"P102"})
-[ :CALLED {
  timestamp:"2026-08-11T09:10:00",
  source_record_id:"CDR-023",
  confidence:0.94,
  evidence_class:"OBSERVED"
}]->
(:Phone {id:"PH07"})
```

8.2 PostgreSQL schema (Plan)

Table	Columns
users	id, name, role_id, status
roles	id, name
cases	id, title, category, status, created_at, owner_id
documents	id, case_id, source_type, filename, hash, uploaded_by, created_at
evidence	id, document_id, record_id, evidence_class, confidence, reliability
audit_logs	id, user_id, action, object_type, object_id, timestamp
notes	id, case_id, author_id, body, created_at

8.3 Inconsistencies in the plan that need a decision

- **Transaction modelling.** Transaction is a node type and TRANSFERRED is a relationship type. The plan does not say whether a transfer is a node, an edge or both (D10).
- **Call modelling.** The example edge is Person CALLED Phone, but the CDR schema records a caller and receiver. How CDR rows and phone ownership (OWNED) map to edges is not stated (D10).
- **Time.** The innovation portfolio says nodes and edges carry validity windows. The edge property table has a single timestamp (D12).
- **Reliability.** The property table lists reliability, but the example edge omits it.
- **Evidence class.** Only OBSERVED is given. The full vocabulary, its mapping to FACT, INFERENCE and RECOMMENDATION, and the scales and assignment of confidence and reliability are not stated (D11).
- **Link between stores.** The field names suggest edge.source_record_id points to evidence.record_id. The plan does not state this join (D16).
- **Missing tables.** The plan requires merge provenance, competing claims, lead and priority records, reports, model and version metadata, and ingestion runs. None has a table in the listed schema (D16).
- **Scope of the graph.** The API is case-scoped for graphs but not for entities. Whether the graph is one graph per case or one global graph is not stated (D13).

9. Graph intelligence

The analytics layer produces structural signals, anomalies, temporal changes, hidden-link candidates and conflicts, then combines them into an explainable investigation priority. No single signal is evidence of criminality.

9.1 Centrality and communities

Centrality runs on Neo4j GDS or NetworkX. Communities use Louvain (Blondel et al., 2008), which the plan uses to reduce graph complexity. Leiden and dynamic communities are roadmap.

Metric	Meaning	Use
Degree	Number of direct connections	Highly connected entities
Betweenness	Frequency on shortest paths	Bridge and intermediary detection
Closeness	Average distance to the network	Reach
Eigenvector	Connected to important nodes	Influential neighbourhood
PageRank	Importance from incoming relationships	Ranked influence

The UI must show the underlying records and context next to any metric. Which algorithms run in GDS and which in NetworkX is not specified (D23).

9.2 Anomaly detection (MUST)

The method is Isolation Forest (scikit-learn) with baselines. The plan's examples of what it should surface are a recent communication spike and anomalous transactions. Planted anomalies in the synthetic data are the test set, scored with Precision@K and false-positive rate.

The input features, the baseline definition (per entity, per case or per time window) and the model parameters are not specified (D7).

9.3 Temporal analysis and "What Changed?"

The investigator compares a baseline window with an incident window. The plan offers 7-day, 30-day, 90-day and custom windows, using snapshot and delta analysis.

flowchart TD

```
A["Baseline window vs incident window"] --> B["New nodes, new edges,<br/>frequency change"]
B --> C["Anomalies + network evolution"]
C --> D["Evidence-backed explanation"]
```

The engine highlights: newly created edges, increased communication frequency, new locations, newly active entities, transaction changes and community shifts. The plan's demo also uses the 48 hours before the incident. Priority of this feature is decision D2, and its dependence on the time model is decision D12.

9.4 Hidden-link discovery

The system ranks plausible missing relationships as leads for human review. It must never silently convert a prediction into a fact. The MVP method is common neighbours, paths and temporal evidence; node2vec and GNN are roadmap.

Signal	Example	Result
Common neighbours	A and B both repeatedly connect to C	Candidate shared network
Temporal synchronisation	A and B activity spikes in the same narrow period	Potential coordination signal
Shared locations	Repeated proximity near relevant events	Potential co-location lead
Transaction path	A to X to B through intermediaries	Potential financial relationship
Community overlap	Two clusters share an emerging connector	Potential bridge
Identifier overlap	Same phone, vehicle or account appears across records	Potential association

Planted hidden links are scored with Precision@K and Recall@K for K = 5, 10 and 20. How the signals are combined into one score, and the cut-off for showing a candidate, are not specified (D9).

9.5 Contradiction detection (NICE)

The system stores competing claims with their provenance and flags the conflict. It does not decide which source is correct.

Claim A	Claim B	Action
A report places person A at location X	A location record places a device at Y at the same time	Flag a possible conflict
A record says an alias belongs to A	Another record maps the alias to B	Flag identity ambiguity
A vehicle is associated with A	The registration record associates it with B	Flag an ownership conflict

The tolerance for "same time" and the storage of competing claims are not specified (D16). The demo script includes a conflicting-location beat (section 13), so the classification of this feature as NICE is part of D2.

9.6 Explainable investigation priority (MUST)

Each entity gets a weighted score built from six components.

```
\text{Priority} = w_1\, \text{network\_influence} + w_2\, \text{relationship\_strength} + w_3\, \text{recency} + w_4\, \text{anomaly\_score} + w_5\, \text{temporal\_relevance} + w_6\, \text{evidence\_confidence}
```

The output is a label such as "Investigation Priority: HIGH" with an explanation, never a criminal probability. The plan's example explanation: "HIGH because the entity has 12 strong links, 2 bridge relationships, a recent communication spike, 3 anomalous transactions and supporting evidence from 3 independent source types."

The weights, the scaling of each component, the definition of "strong link" and the HIGH / MEDIUM / LOW bands are not specified (D8).

10. Investigator intelligence: leads, copilot, reports

This layer presents findings to people. Every output is labelled FACT, INFERENCE or RECOMMENDATION, links back to source evidence, and leaves the decision with the investigator.

10.1 Leads and labelling (Plan)

The dashboard surfaces these review items: high-priority leads, new relationships, anomalies, evidence conflicts and emerging communities. Each lead shows its priority, the explanation (section 9.6) and the evidence behind it.

The labelling rule separates what was observed (FACT), what the model inferred (INFERENCE) and what is suggested for review (RECOMMENDATION). How these three labels relate to the stored evidence_class values is decision D11.

The presentation's stakeholder flow is: the investigator opens a lead, the AI engine extracts and scores priority, the intelligence analyst cross-validates through the graph, and a senior officer reviews an evidence-backed brief.

10.2 Investigation copilot (MUST)

The copilot is a controlled, read-only assistant. It must not allow arbitrary LLM-generated database commands.

flowchart TD

```
A["User question"] --> B["Intent classification"]
B --> C["Allowed operation"]
C --> D["Parameter validation"]
D --> E["Read-only query template"]
E --> F["Graph / database retrieval"]
F --> G["Evidence retrieval"]
G --> H["Grounded response"]
H --> I["Evidence references"]
I --> J["Uncertainty check"]
```

Minimum question set (Plan, the proposal's ten examples):

1. Show people connected to Case 102 within two hops.
2. Why is Person X marked high priority?
3. Which relationships appeared during the last 30 days?
4. What changed in the 48 hours before the incident?
5. Find transaction paths between Entity A and Entity B.
6. Show communication spikes before the incident.
7. Which people connect otherwise separate communities?
8. Show evidence supporting the connection between A and B.
9. Show contradictions in the current evidence.
10. Summarise this case while separating facts from inferences.

Security controls (Plan):

- Uploaded documents are untrusted data.
- Tools are allowlisted.
- Database permissions for the copilot are read-only.
- Parameters are validated.
- Retrieved evidence is kept separate from system instructions.
- Every claim requires an evidence reference.
- When evidence is missing, the copilot states that and refuses to invent an answer.

Not specified (D3): the LLM provider and model, whether it runs hosted or local, whether the LLM performs intent classification, answer writing or both, the list of allowed operations and their query templates, and how prompt-injection and hallucination tests (section 13) are pass/fail scored.

10.3 Intelligence report (MUST)

The report contains: case metadata and scope; executive summary; key entities and communities; important connectors; timeline and major events; anomalies and changes; the evidence supporting each

finding; contradictions and unresolved claims; confidence and source reliability; investigator notes and next-review items; audit metadata and model version. Conclusions carry the FACT / INFERENCE / RECOMMENDATION label.

The output format (for example PDF, DOCX or HTML), whether reports are stored, and who may generate them are not specified (D18, D14).

11. Interfaces: API and UI

The plan lists 12 API endpoints and 9 UI screens. It gives no request or response schemas, and lists no endpoint for several MUST features.

11.1 API endpoints (Plan)

Method	Endpoint	Purpose
POST	/api/cases	Create case
GET	/api/cases/{id}	Case details
POST	/api/cases/{id}/documents	Upload evidence
POST	/api/ingestion	Start ingestion
GET	/api/cases/{id}/graph	Case graph
GET	/api/entities/{id}	Entity profile
GET	/api/entities/{id}/neighbors	Neighbourhood
GET	/api/cases/{id}/timeline	Timeline
GET	/api/cases/{id}/anomalies	Anomalies
POST	/api/copilot/query	Grounded query
GET	/api/cases/{id}/reports	Reports
GET	/api/audit	Audit logs

Contracts are typed with Pydantic (Plan). The UI filters imply these graph query inputs (Derived): entity type, relationship type, source, time range and hop depth of 1, 2 or 3. Exact parameter names, request bodies, response shapes, pagination and error formats are not specified.

Endpoints the MUST and NICE features need but the plan does not list (D15): login and session, entity-resolution review and correction, hidden-link candidates, contradictions, "What Changed?" comparison, evidence detail for the evidence drawer, report generation, notes, and user and role administration.

11.2 UI screens (Plan)

Screen	Purpose	Key components
--------	---------	----------------

Screen	Purpose	Key components
Landing	Explain value immediately	Hero, problem, solution, innovation, demo call-to-action
Command Center	Case-level overview	Cases, alerts, anomalies, new relationships, priority leads
Case Workspace	Primary investigation environment	Summary, evidence, graph, timeline, transactions, notes
Network Explorer	Relationship analysis	Graph, filters, path finder, communities, evidence drawer
Timeline	Temporal reconstruction	Time slider, event lanes, before/after comparison
Entity Profile	360-degree entity view	Identifiers, relationships, activity, evidence, conflicts
Copilot	Natural-language analysis	Query, grounded response, result graph or table
Reports	Formal intelligence output	Facts, inferences, evidence, conflicts, audit metadata
Audit	Accountability	User, action, object, timestamp

11.3 Graph interactions (Plan)

- Zoom, pan and search
- 1-hop, 2-hop and 3-hop exploration
- Filter by entity type, relationship, source and time
- Community and key-connector highlighting
- Path finding
- Evidence, confidence and source inspection
- Expand and collapse a neighbourhood
- Compare time windows

The graph-overload controls are case scoping, time, type and source filters, community aggregation, and progressive neighbourhood expansion. The performance target is sub-second latency for common demo queries. This is a target, not a measured result.

11.4 Dashboard elements (Plan)

Active cases, high-priority leads, new relationships, anomalies, evidence conflicts, emerging communities, graph canvas, evidence drawer, timeline.

11.5 Design language (Plan)

A restrained professional intelligence-command-centre look: dark interface, clear hierarchy, graph and map-centred canvas, evidence cards, subtle motion. The plan rules out generic CRUD cards and excessive neon. The graph is a working analytical surface, not a decorative background.

11.6 Optional features

- **Geo-temporal map (NICE):** combines location proximity with time and event context, using MapLibre GL JS or Leaflet (D23).
- **Mission mode (NICE):** "optional structured hypotheses, evidence trails and follow-lead workflow". The plan gives no further detail, so it has no design here.

Supported UI languages and browsers are not specified (D4).

12. Security, privacy and auditability

The plan requires role-based access, a full audit trail and human review of consequential decisions. It states which controls apply "in production" separately, and does not say which of those the prototype includes.

12.1 Controls (Plan)

Area	Control
Access	Role-based access control with four roles: investigator, analyst, supervisor, administrator
Privilege	Least privilege for databases and services; the copilot's database access is read-only
Audit	Every sensitive read and write is logged (audit_logs table, Audit screen, GET /api/audit)
Data	Synthetic or pseudonymised data in public demonstrations
LLM safety	Uploaded text is untrusted; tools are allowlisted; parameters are validated; evidence is separated from instructions (section 10.2)
Responsible AI	FACT, INFERENCE and RECOMMENDATION are separated in the UI; a human reviews before any consequential action; the system never labels a person a criminal or recommends coercive action from an AI score alone
Reproducibility	Model and version metadata are stored
Correction	Authorised users can correct entity-resolution

Area	Control
	mistakes
Compliance	Follow applicable Indian data-protection, evidence-handling and organisational access requirements

The plan does not name the specific laws or rules.

12.2 Stated as production-only (Plan)

- TLS in transit and encryption at rest.
- Secure file validation and malware scanning.

Whether the prototype includes any of these is decision D22.

12.3 Not specified (D14)

- The authentication mechanism (for example sessions or tokens) and password handling.
- The permission matrix: which role may create cases, upload, resolve entities, run the copilot, generate reports, read audit logs and administer users.
- How the six personas in the plan map to the four roles. The personas are investigator, intelligence analyst, cybercrime investigator, financial crime analyst, senior officer and supervisor.
- Which actions count as "sensitive" and are audited.
- Whether access is limited per case or per role only.
- The threat model. The plan lists it as a phase-8 deliverable but gives no content.

13. Testing and validation plan

The prototype is validated against planted ground truth using pytest and Playwright. Every number below is an engineering target; the submission must report measured results, not targets.

13.1 Metrics and targets (Plan)

Area	Metric	Target or reporting
NER	Precision, recall, F1	0.85 or higher on a controlled domain set (see 6.4)
Entity resolution	Precision, recall, F1	0.90 or higher precision on the synthetic benchmark
Hidden links	Precision@K, Recall@K	Report for K = 5, 10, 20
Anomaly	Precision@K, false-positive rate	Evaluate on planted anomalies
Graph	Query latency	Sub-second for common demo queries

Area	Metric	Target or reporting
Copilot	Grounded-answer rate	Every demo claim maps to retrieved evidence
UX	Time-to-insight	Compare a scripted manual workflow with the CrimeLens workflow

The definition of "common demo queries", the data split ratios and the manual baseline for time-to-insight are not specified (D24).

13.2 Test types (Plan)

- Unit tests for parsers, scoring and graph utilities.
- Integration tests for ingestion, graph, analytics and API.
- Synthetic ground-truth tests for entity resolution and hidden links.
- Security tests for authentication, authorisation and file validation.
- Prompt-injection tests with malicious instructions inside documents.
- Hallucination tests where evidence is absent.
- Graph stress tests for dense neighbourhoods.
- End-to-end UI tests for the core investigator journey.
- Regression tests after any model or scoring change.

The core journey for end-to-end tests is: login, case, ingest, validate, extract, resolve, graph, analyse, timeline, leads, evidence, copilot, report, audit.

13.3 Failure-safe behaviour (Plan)

- When evidence is missing, the system says so.
- When models disagree, uncertainty is shown.
- When sources conflict, the conflict is surfaced.
- When the copilot cannot answer from evidence, it refuses to invent an answer.

13.4 Acceptance scenario: the five-minute demo (Plan)

Time	Beat	What the prototype must show
0:00 to 0:30	Problem	FIR, CDR, transaction and report as separate sources
0:30 to 1:15	Ingest	Upload of a synthetic bundle, with validation and extraction
1:15 to 2:00	Graph	The network, with communities and key connectors switched on
2:00 to 2:45	Timeline	The 48 hours before the incident, with new links and an

Time	Beat	What the prototype must show
		anomaly
2:45 to 3:30	Evidence	A bridge entity opened, with its CDR, transaction and report sources
3:30 to 4:15	Copilot	"Why is this entity a priority?" and "What changed?", each with evidence
4:15 to 4:45	Contradiction	Conflicting location evidence and its review state
4:45 to 5:00	Close	A generated brief and the human-in-the-loop explanation

This scenario needs hidden-link discovery, "What Changed?" and contradiction detection to be working, which is why their priority is open (D2).

14. Deployment and environment

The prototype deploys with Docker Compose on local or commodity hardware, without a GPU. The intended production direction is on-premises or private cloud, and is out of scope here.

14.1 Prototype versus production (Plan)

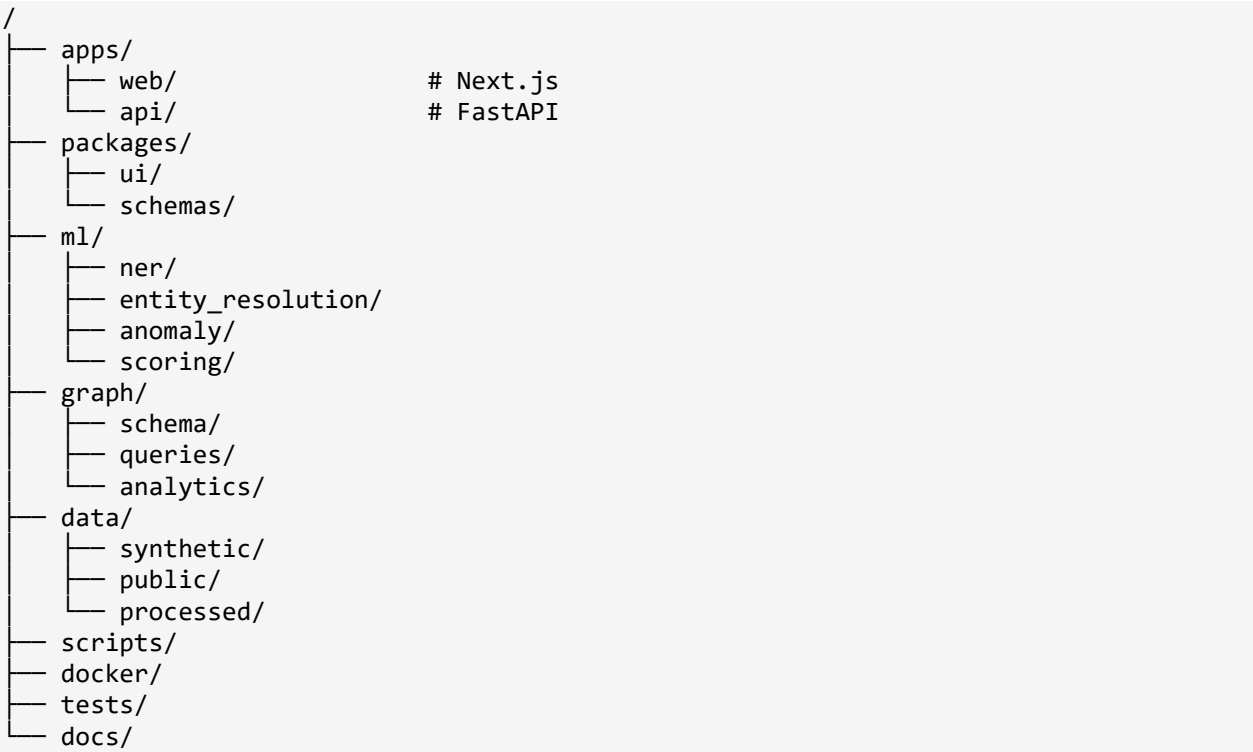
Area	Prototype	Production roadmap (out of scope)
Ingestion	Batch CSV, JSON, PDF, text	Streaming adapters and Kafka
Processing	Python and Pandas	Polars or Spark with orchestration
Search	PostgreSQL and Neo4j	OpenSearch or Elasticsearch
Storage	Local or Docker volume	Encrypted S3-compatible object storage
LLM	Controlled API or local model, as permitted	Approved on-premises model
Deployment	Docker Compose	Private-cloud or on-premises Kubernetes

14.2 Hardware (Plan)

Configuration	CPU	RAM	GPU	Storage
---------------	-----	-----	-----	---------

Configuration	CPU	RAM	GPU	Storage
Minimum demo	4 to 6 cores	16 GB	Not required	50+ GB SSD
Recommended development	8+ cores	32 GB	Optional, 8 to 12 GB VRAM	100+ GB SSD

14.3 Repository structure (Plan)



14.4 Build phases and ownership (Plan)

Phase	Deliverables
1. Data	Schemas, generator, validation, benchmark
2. Backend	FastAPI, PostgreSQL, authentication, APIs
3. Graph	Neo4j schema, ingestion, graph queries, analytics
4. NLP	NER, normalisation, relation extraction, entity resolution
5. Intelligence	Anomaly, temporal change, explainable priority
6. Frontend	Dashboard, graph, timeline, entity and evidence views
7. Copilot	Intent layer, retrieval, safe graph tools

Phase	Deliverables
8. Security	RBAC, audit, validation, threat model
9. Demo	Script, metrics, failure tests, pitch

Team role	Ownership
AI/ML	NER, entity resolution, anomaly detection, explainability
Graph/Data	Neo4j, graph algorithms, synthetic generator
Backend	FastAPI, PostgreSQL, authentication, APIs
Frontend	Next.js, graph, timeline, UX
Product/UI	Design system, investigator workflow, demo polish
Security/DevOps	RBAC, audit, Docker, testing

14.5 Not specified (D23)

- The list of Docker Compose services. The stack implies a web app, an API, PostgreSQL and Neo4j (Derived); where NLP and ML workers run, and where the LLM runs, is not stated.
- The Neo4j edition and how GDS and NetworkX divide the analytics work.
- The map library: the proposal says MapLibre GL JS or Leaflet, the presentation shows MapLibre.
- Configuration, secrets handling and seed-data loading.

15. Risks, open questions and items to confirm

Twenty-four details that the prototype needs are not specified in either source. They are listed below and are referenced by ID throughout this document.

15.1 Decisions needed

ID	Decision needed	Affects
D1	Product name: CrimeLens (presentation) or NETRA-X (proposal)	Whole document
D2	Priority of hidden-link discovery, "What Changed?" and the Landing screen, none of which is in the MUST / NICE / FUTURE table; and whether contradiction detection stays	2, 9, 11, 13

ID	Decision needed	Affects
	NICE although the demo uses it	
D3	LLM provider and model, hosted or local, its exact role, and the list of allowed copilot operations with their query templates	4, 10
D4	NER model or models, supported languages for extraction and UI, supported browsers	6, 11
D5	Relation label set, relation classifier and its training data, list of event types	6
D6	Entity resolution: normalisation rules, fuzzy-match algorithm, thresholds, the rule for merge versus review, use of graph evidence, role of Fellegi and Sunter	7
D7	Anomaly detection: input features, baseline definition, Isolation Forest parameters	9
D8	Priority score: weights, scaling of components, definition of a strong link, HIGH / MEDIUM / LOW bands	9
D9	Hidden-link scoring: how signals combine, and the cut-off for showing a candidate	9
D10	Graph modelling: transaction as node, edge or both; how CDR rows and phone ownership become edges	8
D11	Evidence class vocabulary, mapping to FACT / INFERENCE / RECOMMENDATION, scales and assignment of confidence and	8, 10

ID	Decision needed	Affects
	reliability	
D12	Time model: validity windows or a single timestamp per edge	8, 9
D13	Graph scope: one graph per case or one global graph; whether entity resolution crosses cases	7, 8
D14	Authentication mechanism, permission matrix for the four roles, mapping of the six personas, audited actions, per-case access	10, 12
D15	API endpoints for login, resolution review, hidden links, contradictions, "What Changed?", evidence detail, report generation, notes, user administration; request and response schemas	11
D16	PostgreSQL tables for merge provenance, competing claims, leads and scores, reports, model versions, ingestion runs; the join between edges and evidence rows	6, 7, 8, 9
D17	Validation rules, PDF handling, upload limits, schemas for surveillance and OSINT, mapping of cell_id to location, how an ingestion run executes	3, 5
D18	Report output format and storage	10
D19	Anonymisation method in the pipeline	5
D20	Which public datasets are used and under which licences	5

ID	Decision needed	Affects
D21	Synthetic generator parameters: call, FIR and narrative counts, time span, incident date, random seed, split ratios	5
D22	Which production-only controls (TLS, encryption at rest, malware scanning) the prototype includes	12
D23	Compose services, Neo4j edition, GDS versus NetworkX split, map library, configuration and secrets	4, 9, 14
D24	Evaluation protocol: definition of common demo queries, split ratios, manual baseline for time-to-insight	13

15.2 Risk register (Plan)

Risk	Impact	Planned mitigation
Poor data quality	Wrong or missing fields	Validation and source quality
Entity ambiguity	False merges and splits	Multi-stage matching and human review
Multilingual text	NER errors	Language detection and multilingual models
Graph explosion	Slow, cluttered UI	Case scope, time filters, communities, progressive expansion
False positives	Loss of trust	Evidence-backed leads, not guilt labels
LLM hallucination	Wrong intelligence	Retrieval grounding and tool constraints
Prompt injection	Model manipulation	Untrusted-data isolation and allowlists
Conflicting sources	Bad conclusions	Preserve claims and raise

Risk	Impact	Planned mitigation
		contradiction flags
Data drift	Model degradation	Monitoring and re-evaluation
Privacy leakage	Sensitive exposure	RBAC, encryption, auditing
Legacy systems	Integration friction	Adapter and API layer with a canonical schema
Limited compute	Slow NLP	Small models, batching, CPU-first design
Bias	Unequal errors	Representative evaluation and human review
Incomplete evidence	Overconfidence	Explicit uncertainty and missing-data indicators

The plan does not say which of these mitigations are built in the prototype. Data drift monitoring and legacy-system adapters, for example, have no corresponding component in sections 3 to 12.